

Deployment Guide

ExamForge AI | Generated: 2026-08-02

Prerequisites

Before deploying the ExamForge AI application, the following prerequisites must be met. This guide assumes deployment to a Node.js-compatible hosting environment (Vercel, AWS, Docker, or similar) with access to the Supabase project.

Prerequisite	Status	Notes
Node.js 18+	Required	For Next.js 16 runtime
Supabase project	Active	Project ref: pzfnptrrnxkgodclyhft
Environment variables	Required	NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY
Flutterwave credentials	Required	For payment processing
Production build	NOT VERIFIED	Must pass npm run build without errors
CSP headers	NOT IMPLEMENTED	Critical security requirement
Rate limiting	NOT IMPLEMENTED	Critical for API protection
Monitoring	NOT IMPLEMENTED	Sentry/OpenTelemetry recommended

Pre-Deployment Checklist

The following items MUST be completed before production deployment. Items marked as FAIL or NOT VERIFIED in the Production Checklist must be resolved. The most critical blockers are listed below with their remediation steps.

1. Security Remediation

Add Content Security Policy headers in middleware.ts or next.config.ts. The CSP should allow scripts from self and the Supabase domain, styles from self and inline, images from self and Supabase storage, and connections to the Supabase API and WebSocket endpoints. Add authorization checks to all server actions (verify user identity and role before performing mutations). Validate the auth callback next parameter to prevent open redirects. Add rate limiting to API routes using Redis-backed rate limiting (e.g., Upstash) or an in-memory rate limiter.

2. Data Scoping Remediation

Fix all queries that do not properly scope data by school or teacher. The getSchoolAdminStats, getTeacherStats, and reports service queries must filter by school_id or teacher exam IDs. Fix the teacher dashboard to count students instead of exams. Fix the analytics date range filter to apply the computed dateFilter to the query. Add pagination to all list queries (schools, students, teachers, parents, marketplace, CBT exams).

3. Middleware RBAC Remediation

Update the ROLE_ROUTE_PREFIXES in middleware.ts to match the actual route structure. The current prefixes (/admin, /school, /teacher, /student) do not match the actual routes (/dashboard/super-admin, /dashboard/school-admin, /dashboard/teacher, /dashboard/student).

Either update the prefixes or restructure the routes to match the guard prefixes.

Environment Configuration

The following environment variables are required for production deployment. Sensitive values should be stored in the hosting platform's secret manager, not in .env files.

Variable	Required	Description
NEXT_PUBLIC_SUPABASE_URL	Yes	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	Supabase anonymous key (public)
SUPABASE_SERVICE_ROLE_KEY	Server only	For admin operations (NEVER expose to client)
FLUTTERWAVE_PUBLIC_KEY	Yes	Flutterwave payment public key
FLUTTERWAVE_SECRET_KEY	Server only	Flutterwave payment secret key
FLUTTERWAVE_WEBHOOK_HASH	Server only	For webhook signature verification
SENTRY_DSN	Recommended	For error tracking
NEXT_PUBLIC_SENTRY_DSN	Recommended	For client-side error tracking

Build and Deploy

Build Commands

```
npm run build
```

The build command compiles the Next.js application for production. The output: "standalone" configuration in next.config.ts produces a self-contained build in .next/standalone/ that can be deployed without node_modules. The build script also copies static assets and public files to the standalone directory.

Start Command

```
NODE_ENV=production node .next/standalone/server.js
```

The standalone server starts on port 3000 by default. Set the PORT environment variable to customize. The server should be run behind a reverse proxy (nginx, Cloudflare, etc.) for SSL termination and additional security headers.

Docker Deployment

The standalone output is Docker-ready. Create a Dockerfile that copies the standalone build, installs only production dependencies, and runs the server. Use a multi-stage build to minimize image size. The Docker image should expose port 3000 and set NODE_ENV=production.

Post-Deployment Verification

After deployment, verify the following items to ensure the application is functioning correctly:

Verification	Method	Expected Result
Auth flow works	Login with test account	Redirect to dashboard, session established
RBAC works	Login as different roles	Role-appropriate access only

Verification	Method	Expected Result
Data scoping works	Login as school admin	Only see own school data
Realtime works	Open notifications page	Live indicator, new notifications appear
Payments work	Test checkout flow	Flutterwave payment completes
CSP works	Check browser console	No CSP violation errors
Rate limiting works	Send 100 requests to API	Rate limit response after threshold
Error tracking works	Trigger a test error	Error appears in Sentry
Performance acceptable	Run Lighthouse audit	FCP < 2s, LCP < 3s, CLS < 0.1
Accessibility passes	Run axe-core scan	0 Critical/High violations

Monitoring and Alerting

The following monitoring infrastructure should be set up before production deployment:

Error Tracking: Configure Sentry for both server-side and client-side error tracking. Set up alerts for error rate spikes (>1% of requests). Use the Sentry Next.js SDK for automatic error capture and source map upload.

Performance Monitoring: Configure OpenTelemetry for distributed tracing. Set up alerts for slow API responses (>500ms p99) and high error rates. Use the Next.js instrumentation hook to initialize tracing.

Uptime Monitoring: Set up health checks on the /api/health endpoint (create if needed). Configure alerts for downtime (>5 minutes). Use a service like Pingdom or UptimeRobot.

Analytics: Integrate PostHog or similar for product analytics. Track key user flows: login, exam creation, exam taking, result viewing, and payment completion.

Rollback Plan

If critical issues are discovered after deployment, the following rollback procedure should be followed: (1) Revert to the previous deployment using the hosting platform's rollback feature (Vercel: instant rollback, Docker: redeploy previous image). (2) If the issue is a database migration, use Supabase's point-in-time recovery to restore the database to a previous state. (3) If the issue is a Supabase configuration change, revert the change in the Supabase dashboard. (4) Communicate the rollback to stakeholders and document the root cause for the post-mortem.